Национальный исследовательский ядерный университет «МИФИ»
Институт интеллектуальных кибернетических систем
Кафедра криптологии и дискретной математики

Курсовая работа
по дисциплине «Классическое машинное обучение»

Тема работы:
«Эффективное сочетание параметров для создания лекарственных препаратов»

Выполнил: студент группы М25-525
Тепляков А.Н.

Содержание

1. Библиотеки.....4
2. Первичный анализ данных.....5
2.1. Оценка данных:.....5
2.2. Анализ пропусков, нулевых значений и бинарных признаков.....5
2.2.1. Пропуски.....5
2.2.2. Нулевые колонки.....5
2.2.3. Бинарные признаки.....5
2.2.4. Анализ выбросов.....5
2.3. Целевая переменная.....5
3. Анализ корреляции между признаками.....6
3.1. Оценка важности признаков.....6
4. Выводы по исследовательскому анализу данных.....6
5. Обучение моделей.....7
5.1. Регрессия для IC50.....9
5.1.1. Подготовка данных.....9
5.1.2. Предобработка данных.....9
5.1.3. Модели и подбор гиперпараметров.....9
5.1.4. Результаты обучения (на тренировочной выборке).....10
5.1.5. Оценка на тестовой выборке для лучшей модели.....10
5.1.6. Выводы.....10
5.2. Регрессия для CC50.....11
5.2.1. Подготовка данных.....11
5.2.2. Предобработка данных.....11
5.2.3. Модели и подбор гиперпараметров.....11
5.2.4. Результаты обучения (на тренировочной выборке).....12
5.2.5. Оценка на тестовой выборке.....12
5.2.6. Выводы.....12
5.3. Регрессия для SI.....13
5.3.1. Подготовка данных.....13
5.3.2. Предобработка данных.....13
5.3.3. Модели и подбор гиперпараметров.....13
5.3.4. Результаты обучения (на тренировочной выборке).....14
5.3.5. Оценка на тестовой выборке.....14
5.3.6. Выводы.....14
5.4. Классификация: превышает ли значение IC50 медианное значение выборки.....15
5.4.1. Подготовка данных.....15
5.4.2. Предобработка данных.....15

5.4.3. Модели и подбор гиперпараметров....15
5.4.4. Метрики KNN на тестовой выборке....16
5.4.5. Ключевые выводы....16
5.5. Классификация: превышает ли значение CC50 медианное значение выборки....17
5.5.1. Результаты обучения моделей (best_score_accuracy)....17
5.5.2. Метрики RandomForest на тестовой выборке....17
5.5.3. Выводы....17
5.6. Классификация: превышает ли значение SI медианное значение выборки....18
5.6.1. Результаты обучения моделей (best_score_accuracy)....18
5.6.2. Метрики KNN на тестовой выборке....18
5.6.3. Выводы....18
5.7. Классификация: превышает ли значение SI значение 8....19
5.7.1. Лучшая модель: KNN....19
5.7.2. Метрики KNN на тестовой выборке....19
5.7.3. Выводы....19
6. Вывод....20

1. Библиотеки

При проведении исследования «Эффективное сочетание параметров для создания лекарственных препаратов» были переменены следующие основные библиотеки:

1) Для работы с данными:
   a) pandas — загрузка, предобработка и анализ табличных данных (например, чтение Excel-файлов, фильтрация, агрегация).
   b) numpy — работа с массивами и математические операции (не упомянут в коде, но обычно необходим для pandas и sklearn).
2) Для визуализации:
   a) matplotlib.pyplot — построение базовых графиков (гистограммы, boxplot и т.д.).
   b) seaborn — расширенная визуализация (распределения, корреляционные матрицы, категориальные графики).
3) Для машинного обучения и предобработки (scikit-learn):
   a) sklearn.model_selection (train_test_split, GridSearchCV) — разделение данных на обучающую/тестовую выборки, подбор гиперпараметров.
   b) sklearn.preprocessing (StandardScaler) — масштабирование признаков.
   c) sklearn.impute (SimpleImputer) — заполнение пропусков.
   d) sklearn.compose (ColumnTransformer) — комбинирование разных преобразований для колонок.
   e) sklearn.decomposition (PCA) — уменьшение размерности данных.
4) Алгоритмы моделей:
   a) LinearRegression — линейная регрессия;
   b) DecisionTreeRegressor — дерево решений для регрессии;
   c) KNeighborsRegressor — метод k-ближайших соседей для регрессии;
   d) RandomForestRegressor — случайный лес для регрессии.
   e) sklearn.metrics (r2_score, mean_squared_error, mean_absolute_error) — оценка качества регрессионных моделей.
   f) catboost (CatBoostRegressor) — градиентный бустинг для регрессии, эффективный для табличных данных.

2. Первичный анализ данных

Был импортирован массив данных в формате csv.

2.1. Оценка данных:

Количество записей: 1 001.

Количество столбцов: 213.

2.2. Анализ пропусков, нулевых значений и бинарных признаков

2.2.1. Пропуски

Обнаружено три строки с пропусками. Стратегия заполнения - заполнение медианной.

2.2.2. Нулевые колонки

В массиве данных был обнаружен ряд колонок, которые в большой степени до 99% заполнены нулями. Проблемные колонки с мы соберем в отдельный список и удалим перед началом обучения моделей.

2.2.3. Бинарные признаки

Ряд колонок заполнены строго 0 или 1, эти колонки будут исключены из численного преобразования.

2.2.4. Анализ выбросов

Напишем специальную функцию для поиска колонкок с долей выбросов > 20 % с использованием межквартильного размаха (IQR).

При обработки удалсь выявить 8 колонок с высокой долей выбросов (например, Ipc, PEOE_VSA4, VSA_EState10 и др.).

Для борьбы с выбросами применим масштабирование признаков RobustScaler и обучение с помощью «деревянных модели», DesisionTree, RandomForest и CatBoost.

2.3. Целевая переменная

Цель исследование:

- Регрессия для IC50
- Регрессия для CC50
- Регрессия для SI
- Классификация: превышает ли значение IC50 медианное значение выборки
- Классификация: превышает ли значение CC50 медианное значение выборки
- Классификация: превышает ли значение SI медианное значение выборки
- Классификация: превышает ли значение SI значение 8.

Целевые переменные для регрессии имеют большие хвосты в распределении, а вот переменные для классификация IC50, CC50 и SI хорошо сбалансированы, вот SI значение 8 имеет дисбаланс классов..

3. Анализ корреляции между признаками

Ищем пары признаков с корреляцией ≥ 0,95 с помощью написанной нами функции. Мы выявили высококоррелирующие группы признаков (например, MolWt и HeavyAtomMolWt, Chi0 и Chi0v и др.). Косвенно, корреляция подтверждается тем, что название колонок похожи. Соберем эти колонки в отдельный список и удалим перед началом обучения.

3.1. Оценка важности признаков

Для оценки важности признаков мы применим корреляционный анализ с целевой переменной, анализ взаимной информации, и встроенные методы CatBoost.

1) Корреляционный анализ с целевой переменной (plot_corr_with_target):

   Отбор признаков с корреляцией ≥ 0,1.

2) Взаимная информация (plot_mutual_info_classif, plot_mutual_info_regression).

   отбор признаков с взаимной информацией ≥ 0,1.

3) Оценка с помощью CatBoost (catboost_clas_features, catboost_reg_features):

   Использование встроенного метода get_feature_importance для отбора признаков с важностью ≥ 0,5.

4. Выводы по исследовательскому анализу данных

Массив данных содержит пропуски, часть столбцов не содержат полезной информации, есть выбросы и высококоррелирующие признаки. Перед обучением моделей мы должны заполнить пропуски, масштабировать признаки, удалить лишние признаки.Изначальный список признаков мы создадим с применением встроенного метода CatBoost .

5. Обучение моделей

Для обучения модели, мы напишем специальный класс, который принимает на вход список моделей с параметрами для поиска оптимальных гиперпараметров и кроссвалидации, и обучает модель. Успешную модель мы проверяем на тестовой выборке.

```
class ModelFinderGridCV():
    def __init__(self,
                 models_dict,
                 model_scoring,
                 preprocessing):
        self.models_dict = models_dict
        self.model_scoring = model_scoring
        self.preprocessing = preprocessing
        self.best_model_name = None
        self.best_estimator_ = None
        self.best_params_ = None
        self.best_score = -float("inf")
        self.best_params = {}
        self.model_leader_board = []
        self.best_score_name = 'best_score_' + self.model_scoring

    def fit(self, X_train, y_train):

        for name, (model, params) in self.models_dict.items():
            print(f'Обучение {model}.')
            steps = []
            if self.preprocessing is not None:
                steps.append(("preprocessing", self.preprocessing ))
            steps.append(('model',model))

            full_pipeline = Pipeline(steps)

            # оптимизация гиперпараметров
            gridsearch = GridSearchCV(
                estimator=full_pipeline,
                param_grid=params,
                cv=5,
                scoring=self.model_scoring,
                )

                    # Обучение модели
            gridsearch.fit(X_train, y_train)
            self.best_score_name = 'best_score_' + self.model_scoring
            result = {'model_name':name,
                      self.best_score_name :gridsearch.best_score_ ,
                      'best_params':gridsearch.best_params_
                      }
            self.model_leader_board.append(result)

            if gridsearch.best_score_ > self.best_score :
                self.best_score  = gridsearch.best_score_
```

```
            self.best_model_name = name
            self.best_params_ = gridsearch.best_params_
            self.best_estimator_ = gridsearch.best_estimator_

        print(f"{name} - {self.model_scoring}: {gridsearch.best_score_:.4f}")


    def predict(self, X):
        if self.best_estimator_ is None:
            return "Нет обученной модели"
        return self.best_estimator_.predict(X)

    def build_scoreboard(self):
        return pd.DataFrame(self.model_leader_board).sort_values(by = self.best_score_name,
ascending=False).reset_index(drop=True)

    def model_self_test(self, X_test, y_test):
        y_pred = self.best_estimator_.predict(X_test)
        if self.model_scoring == 'r2':
            return {
                "best_model_name": self.best_model_name,
                "r2": r2_score(y_test, y_pred),
                "rmse": mean_squared_error(y_test, y_pred),
                "mae": mean_absolute_error(y_test, y_pred)
            }
        elif  self.model_scoring == 'accuracy':
            return {
            'accuracy':accuracy_score(y_test, y_pred),
            'precision':precision_score(y_test, y_pred),
            'recall':recall_score(y_test, y_pred),
            'f1':f1_score(y_test, y_pred),
            }
```

5.1. Регрессия для IC50

5.1.1. Подготовка данных

Целевая переменная: IC50, mM.

Найдем колонки с долей нулевых значений ≥ 99 % (33 колонки).

Выявим бинарные признаки (30 колонок) , их исключим из масштабирования.

Колонки с выбросами (> 20 %) добавим в список на удаление.

Выявлены высококоррелирующие признаки (47 колонок) с корреляцией ≥ 0,95.

Отобраны важные признаки с помощью CatBoostRegressor (важность ≥ 0,5).

Из списка важных признаков исключены проблемные колонки (пустые, бинарные, с выбросами).

5.1.2. Предобработка данных

Для числовых признаков

1. Заполнение пропусков медианой (SimpleImputer).
2. Масштабирование (RobastScaler).
3. Метод главных компонент (PCA, параметры: 5, 10, 20 компонентов).
4. Разделение данных:
5. Тренировочная выборка: 80 %.
6. Тестовая выборка: 20 %.

random_state=42 для воспроизводимости.

5.1.3. Модели и подбор гиперпараметров

Использованные модели:

- LinearRegression;
- DecisionTreeRegressor;
- KNeighborsRegressor;
- RandomForestRegressor;
- CatBoostRegressor.

Поиск гиперпараметров GridSearchCV с 5-кратной кросс-валидацией, метрика — R^2

5.1.4. Результаты обучения (на тренировочной выборке)

Модель	$R2$ (лучший балл)	Оптимальные параметры
RandomForest	0,3509	`{'model__max_depth': 10, 'model__max_features': 'sqrt', 'model__min_samples_leaf': 2, 'model__min_samples_split': 2, 'model__n_estimators': 200, 'preprocessing__num__pca__n_components': 10}`
CatBoost	0,2840	`{'model__depth': 8, 'preprocessing__num__pca__n_components': 20}`
KNN	0,2751	`{'model__n_neighbors': 15, 'model__p': 2, 'model__weights': 'distance', 'preprocessing__num__pca__n_components': 20}`
DecisionTree	0,2554	`{'model__max_depth': 5, 'model__min_samples_leaf': 4, 'model__min_samples_split': 5, 'preprocessing__num__pca__n_components': 10}`
LinearRegression	0,1171	`{'model__fit_intercept': True, 'preprocessing__num__pca__n_components': 20}`

5.1.5. Оценка на тестовой выборке для лучшей модели

Метрики для RandomForest:

Метрика	Значение
Лучшая модель (`best_model_name`)	RandomForest
$R2$	0,4067
RMSE	197 889,83
MAE	239,21

5.1.6. Выводы

RandomForest оказалась лучше всех. Значительное расхождение между RMSE и MAE подтверждает наличие выбросов, влияющих на квадратичные ошибки.

5.2. Регрессия для CC50

5.2.1. Подготовка данных

Целевая переменная: CC50, mM.

Данные были подготовлены, аналогично, разделу 5.1.1.

5.2.2. Предобработка данных

Для числовых признаков

7. Заполнение пропусков медианой (SimpleImputer).
8. Масштабирование (RobastScaler).
9. Метод главных компонент (PCA, параметры: 5, 10, 20 компонентов).
10. Разделение данных:
11. Тренировочная выборка: 80 %.
12. Тестовая выборка: 20 %.

random_state=42 для воспроизводимости.

5.2.3. Модели и подбор гиперпараметров

Использованные модели:

- LinearRegression;
- DecisionTreeRegressor;
- KNeighborsRegressor;
- RandomForestRegressor;
- CatBoostRegressor.

Поиск гиперпараметров GridSearchCV с 5-кратной кросс-валидацией, метрика — R^2

5.2.4. Результаты обучения (на тренировочной выборке)

Модель	$R2$ (лучший балл)	Оптимальные параметры
RandomForest	0,471	{'model__max_depth': 15, 'model__max_features': 'sqrt', 'model__min_samples_leaf': 1, 'model__min_samples_split': 2, 'model__n_estimators': 200, 'preprocessing__num__pca__n_components': 10}
CatBoost	0,457	{'model__depth': 8, 'preprocessing__num__pca__n_components': 20}
KNN	0,4241	{'model__n_neighbors': 10, 'model__p': 2, 'model__weights': 'distance', 'preprocessing__num__pca__n_components': 10}
DecisionTree	0,304	`{'model__max_depth': 3, 'model__min_samples_leaf': 4, 'model__min_samples_split': 2, 'preprocessing__num__pca__n_components': 10}`
LinearRegression	0,24	{'model__fit_intercept': True, 'preprocessing__num__pca__n_components': 20}"

5.2.5. Оценка на тестовой выборке

Метрики для RandomForest:

Метрика	Значение
Лучшая модель (`best_model_name`)	RandomForest
$R2$	0,54
RMSE	237810
MAE	310

5.2.6. Выводы

RandomForest оказалась лучше всех. Значительное расхождение между RMSE и MAE подтверждает наличие выбросов, влияющих на квадратичные ошибки.

5.3. Регрессия для SI

5.3.1. Подготовка данных

Целевая переменная: SI.

Данные были подготовлены, аналогично, разделу 5.1.1.

5.3.2. Предобработка данных

Для числовых признаков

13. Заполнение пропусков медианой (SimpleImputer).
14. Масштабирование (RobastScaler).
15. Метод главных компонент (PCA, параметры: 5, 10, 20 компонентов).
16. Разделение данных:
17. Тренировочная выборка: 80 %.
18. Тестовая выборка: 20 %.

random_state=42 для воспроизводимости.

5.3.3. Модели и подбор гиперпараметров

Использованные модели:

- LinearRegression;
- DecisionTreeRegressor;
- KNeighborsRegressor;
- RandomForestRegressor;
- CatBoostRegressor.

Поиск гиперпараметров GridSearchCV с 5-кратной кросс-валидацией, метрика — R^2

5.3.4. Результаты обучения (на тренировочной выборке)

Модель	$R2$ (лучший балл)	Оптимальные параметры
LinearRegression	−0,0228	`{'model__fit_intercept': True}`
KNN	−0,1524	`{'model__n_neighbors': 15, 'model__p': 1, 'model__weights': 'uniform'}`
RandomForest	−0,1698	`{'model__max_depth': 15, 'model__max_features': 'sqrt', 'model__min_samples_leaf': 2, 'model__min_samples_split': 5, 'model__n_estimators': 200}`
DecisionTree	−0,2065	`{'model__max_depth': 3, 'model__min_samples_leaf': 1, 'model__min_samples_split': 10}`
CatBoost	−0,2235	`{'model__depth': 10}`

5.3.5. Оценка на тестовой выборке

Метрика	Значение
Лучшая модель (`best_model_name`)	LinearRegression
$R2$	0,0078
RMSE	1993203
MAE	217

5.3.6. Выводы

Модель отработала очень плохо. Возможно есть проблема в данных. Дополнительная подготовка и отбраковка признаков, явных результатов не дали

5.4. Классификация: превышает ли значение IC50 медианное значение выборки

5.4.1. Подготовка данных

Целевая переменная: IC50.

Данные были подготовлены, аналогично, разделу 5.1.1.

5.4.2. Предобработка данных

Для числовых признаков

1. Заполнение пропусков медианой (SimpleImputer).
2. Масштабирование (RobastScaler).
3. Метод главных компонент (PCA, параметры: 5, 10, 20 компонентов).
4. Разделение данных:
5. Тренировочная выборка: 80 %.
6. Тестовая выборка: 20 %.

random_state=42 для воспроизводимости.

5.4.3. Модели и подбор гиперпараметров

Использованные модели:

- LinearRegression;
- DecisionTreeRegressor;
- KNeighborsRegressor;
- RandomForestRegressor;
- CatBoostRegressor.

Поиск гиперпараметров GridSearchCV с 5-кратной кросс-валидацией, метрика — accuracy

Модель	Точность (best_score_accuracy)
KNN	0,7250
RandomForest	0,7175
CatBoost	0,7100
LogisticRegression	0,6825
DecisionTree	0,6725

Лучшая модель: KNN.

5.4.4. Метрики KNN на тестовой выборке

Accuracy (общая точность)	0,7114
Precision (точность предсказания положительного класса)	0,7255
Recall (полнота)	0,7115
F1-score (баланс precision и recall)	0,7184

5.4.5. Ключевые выводы

KNN — лучшая модель по результатам кросс-валидации (точность 0,7250).На тестовой выборке модель показывает стабильный рещультат по всем четырем метрикам.

5.5. Классификация: превышает ли значение CC50 медианное значение выборки

Краткий отчёт по результатам обучения моделей

5.5.1. Результаты обучения моделей (best_score_accuracy)

Модель	Точность
RandomForest	0,7213
CatBoost	0,7100
KNN	0,7100
LogisticRegression	0,6950
DecisionTree	0,6938

Лучшая модель по результатам кросс-валидации: RandomForest.

5.5.2. Метрики RandomForest на тестовой выборке

Accuracy (общая точность)	0,7015
Precision (точность предсказания положительного класса)	0,6410
Recall (полнота)	0,3521
F1-score (баланс precision и recall)	0,4545

5.5.3. Выводы

RandomForest — лучшая модель по результатам кросс-валидации (точность 0,7213).

Есть небольшое падение accuracy на тестовой выборке Слабые оценки на Recall и F1-score говорят о том, что модель плохо находит положительные примеры..

5.6. Классификация: превышает ли значение SI медианное значение выборки

5.6.1. Результаты обучения моделей (best_score_accuracy)

Модель	Точность
KNN	0,6813
RandomForest	0,6725
CatBoost	0,6650
DecisionTree	0,6400
LogisticRegression	0,6213

Лучшая модель: KNN.

5.6.2. Метрики KNN на тестовой выборке

Accuracy (общая точность)	0,6667
Precision (точность предсказания положительного класса)	0,6413
Recall (полнота)	0,6344
F1-score (баланс precision и recall)	0,6378

5.6.3. Выводы

KNN — лучшая модель по результатам кросс-валидации (точность 0,6813).и на тесте (с 0,6813 до 0,6667).

5.7. Классификация: превышает ли значение SI значение 8

5.7.1. Лучшая модель: KNN

Точность на этапе кросс-валидации — 0,6813
Результаты всех моделей (по убыванию accuracy):

KNN	0,6813;
RandomForest	0,6725;
CatBoost	0,6650;
DecisionTree	0,6400;
LogisticRegression	0,6213.

Лучшая модель: KNN.

5.7.2. Метрики KNN на тестовой выборке

Accuracy	0,6667;
Precision	0,6413;
Recall	0,6344;
F1-score	0,6378.

5.7.3. Выводы

KNN показал наилучший результат на этапе подбора гиперпараметров.

На тестовой выборке точность модели немного снизилась до 0,6667.

Значение всех метрик близки.

6. Вывод

Работа посещена поиску наиболее эффективного сочетания параметров для создания лекарственных средств.

Мы проанализировали массив данных с использованием инструментов из пакета pandas и sklearn. Мы провели исследовательский анализ данных, обнаружили выбросы, пропуски и коррелирующие между собой признаки.

Далее мы сформировали пайплайн, в котором, заполняли пропуски, масштабировали призанки и применяли метод главных компонент для предобработки данных.

Для задачи регрессии и классификации мы обучали модели семейства KNN,RandomForest

CatBoost, DecisionTree и LogisticRegression.

Задача	Лучшая модель	Метрика	Результаты
Регрессия для IC50	RandomForest	R^2	0,4067
Регрессия для CC50	RandomForest	R^2	0,54
Регрессия для SI	RandomForest	R^2	0,0078
Классификация: превышает ли значение IC50 медианное значение выборки	KNN	accuracy	0,7250
Классификация: превышает ли значение CC50 медианное значение выборки	RandomForest	accuracy	0,7015
Классификация: превышает ли значение SI медианное значение выборки	KNN	accuracy	0,6667
Классификация: превышает ли значение SI значение 8	KNN	accuracy	0,6667

В задачах классификации случайный KNN показал себя лучше чем остальные модели, а в регрессонных лучший - случайный лес.